Laboratorio 3 — Introducción a React

Profesor: Matías Toro
Ayudantes: Benjamín Indo, Benjamín Madrid, Camilo Huerta, Daniel Ramírez, Julián Ferreira, Sebastián Bustos, Sebastián González, Tomás Méndez, Tomás Ubilla

El objetivo de este laboratorio es poner en práctica los conceptos vistos en la clase introductoria de React.js: componentes, manipulación de estados, estados complejos, etc.

Encontrarán la template para este laboratorio en el siguiente enlace. Se siguen las mismas instrucciones que en el laboratorio anterior: Tanto si quiere trabajar en grupo o de forma individual, para aceptar la tarea debe ponerle un nombre a su grupo. El resto de integrantes, de haber, deberá unirse a ese grupo para que todos ganen acceso al repositorio.

Recuerde que debe de instalar las dependencias, yendo a la carpeta del proyecto con `cd lab3` y haciendo `npm install`. Pueden ejecutar el proyecto con `npm run dev`.

En este laboratorio se volverá a la temática de los *Bakemon*. Deberán realizar una página para visualizar información sobre los *Bakemon*, de nombre *Bakedex*.

Por último, **antes de comenzar a desarrollar el laboratorio, cree una rama con el nombre: `lab-3`**.

Para realizar esta tarea, tienen disponible dos fuentes de información. Primero, tienen un archivo `bakemon.ts` con la siguiente estructura:

```
{
  "id": 1,
  "name": "bulbasaur",
  "abilities": [
    {
      "name": "overgrow",
      "is_hidden": false,
      "slot": 1
    },
    {
      "name":
"chlorophyll",
      "is_hidden": true,
      "slot": 3
    }
  ],
  "height": 7,
  "weight": 69,
  "moves": [
    13,
    14,
    15,
    ...
  ],
  "stats": [
    {
      "name": "hp",
      "base_value": 45,
      "effort": 0
    },
    {
      "name": "attack",
      "base_value": 49,
      "effort": 0
    },
    {
      "name": "defense",
      "base_value": 49,
      "effort": 0
    },
    {
      "name": "special-
attack",
      "base_value": 65,
      "effort": 1
    },
    {
      "name": "special-
defense",
      "base_value": 65,
      "effort": 0
    },
    {
      "name": "speed",
      "base_value": 45,
      "effort": 0
    }
  ],
  "types": [
    {
      "type": "grass",
      "slot": 1
    },
    {
      "type": "poison",
      "slot": 2
    }
  ]
},
```

Segundo, el archivo `moves.ts` con la siguiente estructura:

```
{
  "id": 14,
  "name": "swords-dance",
  "damage_class": "status",
  "power": null,
  "pp": 20,
  "priority": 0,
  "stat_changes": [
    {
      "stat": "attack",
      "change": 2
    }
  ],
  "target": "user",
  "type": "normal",
  "ailment": "none",
  "effect_entry": "Raises the user's Attack by two stages."
},
```

Antes de empezar, defina los tipos de Typescript que vaya a necesitar (`Bakemon`, `BakemonMove` y los compuestos que estime convenientes) en base a las dos estructuras anteriores. No es una pregunta evaluada en sí misma, pero las preguntas siguientes darán por hecho que ya los tiene.

P1.

En primer lugar, veremos algunas preguntas conceptuales. El objetivo de estas preguntas es aterrizar los conceptos nuevos que introduce React.

a) ¿Qué es un componente en React? ¿En qué se diferencia de una función común de Typescript?
b) ¿Cuál es la diferencia entre `props` y `state`? ¿Quién puede modificar cada uno, y qué ocurre en la página cuando cambian?
c) JSX permite escribir marcado parecido a HTML dentro de un archivo Typescript. ¿Qué diferencia semántica tiene JSX frente a HTML plano? Ejemplifique con `className` y el uso de `{ }` para insertar expresiones.

P2.

Para comenzar con el desarrollo de la *Bakedex* se necesita una página para ver la información de un *Bakemon*.

Cree un componente `BakemonDisplay` que reciba un *Bakemon* como `prop` y muestre su información: nombre y número, tipos (recuerde que puede tener más de uno), estadísticas base y el total, habilidades, altura y peso.

Al mostrar los tipos, estadísticas y habilidades está recorriendo listas dentro de JSX: recuerde asignar una `key` a cada elemento repetido.

P3.

Cree un componente `MovesList` que reciba los movimientos a mostrar por `props`. Como `bakemon.ts` solo guarda los `id`, deberá cruzar esos ids con `moves.ts` para resolver cada movimiento completo.

De cada movimiento muestre: nombre, tipo y potencia (indique si no tiene), más su descripción.

P4.

Se habrá dado cuenta que los movimientos de un *Bakemon* hacen que la página se vea demasiado grande (y eso que se limitó la cantidad de movimientos). Agregue un botón con la opción de «Expandir movimientos», para que, una vez presionado, se muestren los movimientos, y se oculten si se vuelve a presionar.

Necesitará estado local (`useState`) para recordar si los movimientos están expandidos o no, y un manejador de evento en el botón que lo actualice.

P5.

Ahora que ya tiene un componente para visualizar a un *Bakemon*, cree una página donde se mostrará el listado de *Bakemon*.

Al hacer click en un *Bakemon* de la lista, se debe mostrar el componente trabajado en las preguntas anteriores con la información del *Bakemon* respectivo, junto a un botón para volver a la lista. Piense en qué componente debe guardar el estado de cuál *Bakemon* está seleccionado para que tanto la lista como el detalle puedan usarlo.

P6.

Por último, incluya en su aplicación un constructor de equipos de *Bakemon*. Para ello, agregue 6 celdas donde posiblemente irá un *Bakemon*. Luego, en cada ítem de su listado, agregue la opción de «Agregar al equipo» o «Quitar del equipo». Tenga las siguientes consideraciones:

- No se puede agregar al mismo *Bakemon* más de una vez.
- Si el *Bakemon* no está en el equipo, no se debe mostrar el botón para quitarlo.
- No se debe mostrar el botón de agregar cuando ya está lleno el equipo.

El equipo es, otra vez, estado: piense qué estructura usar para representarlo y cómo actualizarla sin mutarla directamente.